PREMISE SELECTION IS NOT ENOUGH: ACTION-CONDITIONAL EVIDENCE ALLOCATION FOR LEAN

Anonymous authors
Paper under double-blind review

ABSTRACT

Premise selection is usually treated as a ranking problem: choose useful lemmas, pass them to a Lean prover, and hope reconstruction succeeds. We argue that this view is missing a second intervention. A selected name must still be compiled into the proof interface that can consume it: Hammer facts, HammerCore inputs, simp lemmas, `solve_by_elim` facts, Aesop fact rules, or Aesop simp rules. We introduce ACE, a typed evidence compiler for *action-conditional evidence allocation* in Lean. The core finding is counterintuitive but repeatable: once the candidate names are fixed, small typed portfolios can recover nearly all tested oracle headroom, while learned routers do not yet beat the fixed typed control. In a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness, the mixed oracle-core/retrieved action grid solves 58/230 kernel-verified goals versus 38/230 for the best single action; a four-retry fixed portfolio reaches 57/230 out of fold. Removing traced proof-core names still leaves a retrieved-only compiler effect: empty Hammer solves 29/230, the best standalone action solves 35/230, and a fixed portfolio reaches the 52/230 typed oracle. To check that the effect is not only a post-hoc fit to the 230-goal matrix, we freeze the retrieved-only four-action portfolio and evaluate two disjoint fresh folds: on 432 replayable holdout goals, empty Hammer solves 30, the best singleton solves 54, the frozen K=4 portfolio solves 67, and the tested-action oracle is 68. Exact Aesop controls further constrain the mechanism: joint oracle-core plus retrieved exposure is best at 38/230, but identity and simps-only controls reach 36/230 and 35/230, so the supported claim is source/interface sensitivity rather than a large pure facts-versus-simps complementarity. ACE is therefore not a new retriever or an adaptive-router victory; it is evidence that premise selection is incomplete until selected names are typed into executable Lean proof actions.

1 INTRODUCTION

Lean hammers and neural theorem provers depend on premise selection: given a proof goal, the system must choose accessible lemmas that make proof search tractable and reconstructable in Lean. Modern systems therefore invest heavily in retrieving better premises before running a backend prover (Yang et al., 2023; Zhu et al., 2025; Gao et al., 2026). This framing is necessary but incomplete. A selected Lean name is not yet an executable proof-search intervention; it must be represented for the tactic that will consume it. The same name may be useful as a Hammer fact, inert as a simp lemma, harmful as an Aesop rule, or useful only after being paired with a different interface.

This interface decision is easy to overlook because it is not the same as ordinary tactic selection. We are not only choosing whether to call Hammer, Aesop, or simplification. We are choosing how a fixed source of evidence should enter each proof mechanism: as facts, simp rules, HammerCore inputs, `solve_by_elim` facts, or multi-channel Aesop programs. Under bounded search, more evidence is also not monotonic. Additional names can improve proof-core coverage while increasing branching, timeouts, or reconstruction ambiguity. Thus a high-recall premise list can still fail if it is compiled into the wrong Lean interface.

We propose ACE, an action-conditional evidence allocator for bounded Lean proof search. Given a replayable theorem context and a candidate set, ACE compiles selected names into typed proof actions and checks each action in Lean. Every comparison starts from the same empty-premise

Hammer attempt and then spends a fixed retry budget. This protocol turns the main question into a reviewer-checkable intervention study: after names have been proposed, how much verified headroom comes from assigning them to the right proof interface?

The empirical answer is surprisingly strong for simple typed portfolios. In a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness, we identify 230 replayable goals from 490 cleaned traced-corpus goals. A mixed source pool of traced oracle-core names and learned retrieved candidates gives a typed action oracle of 58/230, compared with 38/230 for the best single action. A four-retry fixed typed portfolio after `hammer_empty` reaches 57/230 out of fold and 58/230 when fit on all training data. Homogeneous K=4 controls are much weaker: Aesop-only reaches 49/230, HammerCore-only and simplification-only reach 41/230, and Hammer-only reaches 32/230. The effect is therefore typed diversity, not simply more calls to one backend.

We then remove the most obvious source of concern: traced proof-core assistance. In a retrieved-only anchor on the same 230 replayable goals, empty Hammer solves 29/230 and the best standalone action solves 35/230, while a fixed typed portfolio reaches the 52/230 typed oracle out of fold. To test whether this is a post-hoc peculiarity of the 230-goal matrix, we freeze the retrieved-only four-action portfolio and evaluate two fresh disjoint folds with zero overlap against the original goals. Across 432 replayable holdout goals, empty Hammer solves 30, the best singleton solves 54, K=3 and K=4 solve 67, and the tested-action oracle is 68. This does not make ACE a full theorem-proving leaderboard system, but it does give prospective evidence that typed compilation survives beyond the matrix used to discover it.

The mechanism is also narrower than our early experiments suggested. Aesop with oracle-core plus retrieved evidence exposed jointly as facts and simp lemmas proves 38/230 goals, compared with 29/230 for empty Aesop. Exact controls show that this is not a large isolated facts-versus-simps complementarity: identity exposure proves 36/230, simps-only proves 35/230, facts-only proves 32/230, retrieved-only joint exposure proves 34/230, and oracle-core-only joint exposure proves 35/230. The robust readout is that selected evidence has source-dependent and interface-dependent behavior; the paper does not need the stronger channel-complementarity claim.

This paper makes three contributions. First, we formulate action-conditional evidence allocation as a distinct layer after premise retrieval: selected names must be typed into Lean proof interfaces, not only ranked. Second, we implement ACE in a real Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness and show that fixed typed portfolios recover nearly all tested oracle headroom in a mixed-source matrix, a retrieved-only anchor, and frozen fresh-holdout folds. Third, we report the boundaries that keep the claim honest: broad insertion can hurt, strict syntactic filtering does not explain the gains, exact Aesop controls weaken a tempting complementarity story, and logistic/ComplementNB allocators do not yet beat the fixed typed control.

The scope is deliberately narrower than a full theorem-proving leaderboard. The retrieved-only and fresh-holdout experiments test the downstream compiler once candidate names are fixed; they are not official reproductions of LeanSearch v2 or another complete global retrieval system. We also distinguish bridge replay from open-ended proof generation. These boundaries sharpen rather than weaken the claim: premise selection is incomplete until selected evidence is compiled into the Lean interface that can actually use it.

2 Related Work

Premise selection for Lean. Premise selection is a long-standing interface between theorem provers and large formal libraries. LeanDojo introduced infrastructure and benchmarks for retrieval-augmented theorem proving in Lean (Yang et al., 2023). Recent work on Lean hammers studies learned premise ranking and ATP integration for Lean (Zhu et al., 2025), while LeanSearch v2 focuses on global premise retrieval in Lean 4 (Gao et al., 2026). These works motivate our setting and provide the strongest conceptual baselines: retrieve useful premises from a large library before proof search. Our distinction is that ACE studies the next interface decision after retrieval. It asks how selected names should be compiled into Hammer facts, HammerCore inputs, simp lemmas, Aesop fact rules, Aesop simp rules, or tactic-specific local evidence.

Tactic selection and tactic portfolios. TacticToe learns tactic-level proof search in HOL4 and combines tactic prediction with premise selection inside a search procedure (Gauthier et al., 2021). This is the closest conceptual objection to our framing: a typed action grid can look like another tactic portfolio. Our object is different. Prior tactic-selection systems choose which tactic or tactic sequence to try; ACE holds the selected evidence fixed enough to study how the same names should be represented for the consuming interface, including multi-channel assignment inside a single Aesop invocation. The ablations therefore compare fact-only, simp-only, joint, widened, and source-decomposed evidence programs, not only different tactic names.

Proof repair, compiler feedback, and agentic provers. Several recent systems use Lean feedback to repair or generate proof text. APRIL learns to repair Lean proofs from compiler feedback (Wang et al., 2026); APOLLO and OProver frame formal proving as a broader agentic collaboration or whole-proof search problem (Ospanov et al., 2025; Ma et al., 2026). Process-verified reinforcement learning also uses Lean verification as a training signal (Kim & Yun, 2026). ACE is complementary to these systems: it does not edit proof text or search over natural-language proof plans. Its object is the typed action by which selected names are supplied to Lean's existing proof mechanisms.

Search guidance and local context. Search-guidance methods such as LeanProgress predict proof progress to steer theorem proving (Huang et al., 2025), while long-context theorem proving benchmarks such as miniCTX emphasize the importance of local and user-provided context (Hu et al., 2024). These lines share our view that static global retrieval is incomplete. Our experiments focus on a different mechanism: typed exposure of selected evidence under fixed retry budgets. Failure feedback remains useful in our trace-core discovery experiments, but the verified Mathlib replay result shows that interface choice itself can dominate low-capacity adaptive routing.

3 Problem Formulation

Action-conditional evidence allocation under a fixed retry budget. Let g be a Lean goal and let $\mathcal{A}(g)$ be the accessible name set induced by imports, namespaces, declarations, and local context. A typed proof action is a pair $a = (\tau, P)$, where τ is a Lean interface such as Hammer, HammerCore, simplification, `solve_by_elim`, or Aesop, and $P \subset \mathcal{A}(g)$ is the set of names exposed through that interface. More generally, a typed evidence program is an assignment

$$z_{p,\tau} \in \{0,1\}, \qquad \sum_{p \in \mathcal{A}(g)} z_{p,\tau} \le b_\tau,$$

where $z_{p,\tau} = 1$ means that name p is exposed to interface channel τ under channel budget b_τ. The same name may receive multiple interface labels: an Aesop joint facts+simps action is one compiled program, not two independent retries. A compiler $C(z)$ maps this assignment to Lean tactic syntax, such as Hammer facts, HammerCore inputs, simplifier lemmas, `solve_by_elim` facts, Aesop safe facts, and Aesop simp rules. This separates four decisions that are often conflated: candidate provenance, channel eligibility, channel assignment, and outer action scheduling.

An evidence allocator chooses a sequence of typed programs,

$$a_t = (\tau_t, P_t), \qquad |P_t| \le k_t,$$

and runs each action under a search budget B_t. The prover returns either a kernel-verified success or a failure observation F_t, such as a timeout, missing premise signal, type mismatch, typeclass failure, failed simplification, or reconstruction failure. A closed-loop selector is a policy

$$a_{t+1} = \pi(g, a_{\le t}, F_t, H_t),$$

where H_t includes previous attempt history and candidate features. The objective is to maximize verified success under a fixed total retry budget, rather than to maximize static premise recall at a single k. A fixed typed portfolio is one allocator; a learned action router is another. The experiments ask whether interface typing itself creates headroom before claiming that learned routing has solved the allocation problem.

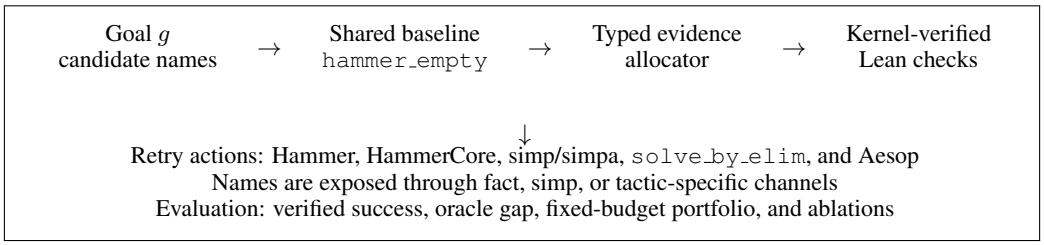

Figure 1: ACE evaluates action-conditional evidence allocation under a fixed retry budget. Selected names are compiled into typed Lean interfaces, and each routed action is checked by the Lean kernel.

Evaluation axes. We separate three notions that are often conflated. *Kernel-verified proof-action success* means that a patched Lean file containing a candidate proof action checks successfully. This is the main verified metric for ACE. *Trace-core success* means that the selected premise set covers the traced proof-core premises for a goal; it is a premise-recovery metric, not a full proving metric. *Bridge verified success* is stricter than trace-core success: trace-core recovery must coincide with successful replay of the original traced tactic script in a real Lean process. Bridge verification is still not a full proof-generation benchmark, because it replays known tactic scripts, but it filters out trace-core cases that drift too far from Lean execution.

4 METHOD

4.1 OVERVIEW

ACE treats a selected name as incomplete until it is assigned to a proof-action interface. Figure 1 gives the full loop. The system first builds a candidate pool and runs a shared empty-premise Hammer attempt. It then spends a fixed retry budget on typed actions that expose selected names as Hammer facts, HammerCore inputs, simp lemmas, `solve_by_elim` facts, or Aesop facts/simp rules. The important design choice is that the retry budget is spent on compiling evidence into interfaces, not on blindly expanding a single untyped premise list.

4.2 TYPED EVIDENCE PROGRAMS

The allocator is built from action families that consume selected names differently. Hammer actions pass names as ordinary premises; HammerCore actions expose a smaller internal proof-search interface; simplification actions use selected names as rewrite/simp lemmas; `solve_by_elim` uses names as local facts; Aesop actions can add names as fact rules, simp rules, or both. For each replayable theorem context, we materialize a patched pre-theorem Lean file for each action and classify the result by running Lean with a fixed timeout.

The fixed portfolio is chosen greedily on the training split and evaluated out of fold. Every policy receives the same first action, `hammer_empty`, and then a retry budget K. This protocol makes the strongest control explicit: an adaptive policy is only useful if it beats or compresses the best fixed sequence of typed actions under the same Lean-call budget. In the current matrix, the train-fitted four-action portfolio after `hammer_empty` is `aesop_core_plus_learned`, `hammerCore_core_plus_learned`, `hammer_core_plus_learned16`, and `solve_by_elim_core`.

This design also prevents a common misreading of the method. The fixed portfolio is not intended to prove that a hand-written order is optimal; it is the hard control that any learned action router must beat. Its near-oracle performance is therefore evidence that interface typing itself is a large effect, while the absence of an adaptive win is reported as a boundary rather than hidden as a negative result.

4.3 AESOP FACT/SIMP EXPOSURE

Aesop is the most important new interface in the verified matrix, so we isolate how selected names enter it. Given a ranked name list, we split available names into fact candidates and simp candidates,

then instantiate Aesop in three exposure modes: facts-only, simps-only, and facts+simps. We also vary source pools: traced oracle-core names, retrieved-only names, and oracle-core plus retrieved names at budgets 8, 16, and 32. This gives a matched source/budget control: the goal and candidate source are fixed, while the channel assignment is removed, added, or widened.

The Aesop ablation is not a new oracle-improving action set; its role is explanatory. The original broad ablation suggested a strong facts+simps interaction, but exact matched controls give a more conservative readout. For the best current oracle-core plus retrieved source, joint exposure recovers the 38/230 Aesop result, yet identity exposure reaches 36/230, simps-only reaches 35/230, facts-only reaches 32/230, and joint-only over exact single-channel controls is only 2 goals. Retrieved-only joint exposure still reaches 34/230 and oracle-core-only reaches 35/230. The supported mechanism is therefore source/exposure/search sensitivity, not a clean causal claim that facts and simps are complementary in isolation.

4.4 Trace-Core Discovery Controllers

The typed-portfolio study was motivated by earlier trace-core controllers. In local Mathlib settings, FAR-Hammer uses visible candidate features and failure types to decide how to rerank or shrink premise sets. In imported-core settings, where lexical and same-file priors are too weak, FAR-Hammer trains a global retriever from traced proof-core labels and then trains a second-stage scorer conditional on the first failure. These controllers are not the verified main method in this draft, but they provide discovery evidence for two ideas that survive in the verified study: premise budgets are non-monotonic, and retry decisions should be evaluated against strong fixed-budget controls.

4.5 Why Typed Actions Should Help

Let $Y_g(z; B) \in \{0, 1\}$ denote the kernel-verified potential outcome for goal g when typed assignment z is checked under search budget B. The marginal intervention effect of exposing name p through interface τ is

$$\Delta_g(p, \tau \mid z, B) = Y_g(z + e_{p,\tau}; B) - Y_g(z; B).$$

This formulation is intentionally intervention-level rather than only a marginal retrieval score. It permits the same name to have positive effect in one interface and negative effect in another, allows facts and simp rules to interact through the current program z, and permits larger evidence sets to have negative marginal effect under bounded search. A typed action helps when the compiler finds assignments whose realized intervention effects are positive under the Lean backend's budget, not merely when a retriever ranks relevant names highly.

5 Experiments

5.1 Protocol

We evaluate three verified evidence layers. The first is a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness over traced-corpus goals. We start from 490 cleaned goals, keep the 230 goals whose original traced tactic script replays in the patched pre-theorem context, and then run typed proof actions in Lean. Success means that the patched Lean file checks successfully.

The second layer removes traced proof-core names from the candidate source and keeps only retrieved candidates. This retrieved-only anchor uses the same 230 replayable goals and the same outer retry protocol, so it isolates whether typed compilation still helps after removing oracle-core assistance.

The third layer is a frozen fresh-holdout check. We take two disjoint 500-goal folds with zero overlap against the original 230-goal set, filter to replayable pre-theorem contexts, and evaluate the predeclared retrieved-only action subset without retuning the portfolio. Fold 0 has 208 replayable goals, fold 1 has 224, and the combined holdout has 432 replayable goals. We treat this as prospective validation of the typed-compiler effect, not as a benchmark-scale population estimate.

We also report trace-core discovery evidence from earlier premise-selection experiments. There, success means that the selected premises cover the proof-core premises extracted from traced proofs. We keep these results because they explain why premise budgets and failure feedback matter, but they

Table 1: Main claim-evidence map. The primary evidence is the verified typed action matrix, retrieved-only anchor, frozen fresh-holdout check, and Aesop channel counterfactual; trace-core and bridge rows are supporting discovery evidence.

Claim	Strong control	Supported result	Readout
Typed actions matter	Best single action: 38/230	Oracle typed grid: **58/230**	+20 goals
Small portfolios are hard controls	Empty Hammer: 29/230	Fixed K=4 OOF: **57/230**	near oracle
Retrieved-only compilation works	Empty Hammer: 29/230; best standalone: 35/230	Fixed K=3/K=4 OOF: **52/230**	reaches typed oracle
Frozen portfolio transfers	Holdout empty Hammer: 30/432; best singleton: 54/432	Frozen K=4: **67/432**; tested oracle: 68/432	prospective check
Aesop exposure is source/interface sensitive	Empty Aesop: 29/230; identity: 36/230	Oracle-core+retrieved joint: **38/230**; retrieved-only: 34/230	modest joint/source effect
Broad insertion can hurt	Oracle-core+retrieved8 facts+simps: 38/230	Oracle-core+retrieved32 facts+simps: 3/230 in original matrix	negative boundary
Adaptive allocation is not yet supported	Fixed K=4 OOF: 57/230	Best learned allocator: 57/230 at K=4	boundary
Failure feedback is useful discovery evidence	Learned+base fallback: 86.3% mean	Trace-core final-base8: 92.9% mean	supporting result
Bridge replay supports premise recovery	Fallback bridge: 44.5%	Final-base8 bridge: 53.5%	supporting result

Table 2: Verified typed proof-action matrix on 230 replayable Mathlib 4.30 traced-corpus theorem contexts. Success means the patched Lean file checks.

Matrix	Attempts	Verified attempts	Best single	Oracle
Original typed grid	3450	246	31/230	51/230
Extended typed grid	7130	506	38/230	58/230
Full Aesop-ablation grid	11500	**582**	**38/230**	**58/230**
Retrieved-only anchor grid	17020	1397	35/230	52/230

are not the main verified claim. BM25 and iterative lexical rows are controlled retrieval policies inside our candidate-pool protocol, not full system-to-system reproductions of the published LeanSearch v2 pipeline.

5.2 Verified Typed Proof-Action Matrix

Table 2 reports the main verified experiment. The original scaled matrix already shows action-dependent headroom: 51/230 oracle versus 31/230 for the best single action. Extending the action set with Aesop and additional typed interfaces raises the oracle to 58/230 and the best single action to 38/230. The full Aesop-ablation matrix does not raise the oracle further, which means the ablation is explanatory rather than a ceiling improvement. The retrieved-only anchor removes traced proof-core names from the candidate source; its lower oracle is expected, but the remaining 52/230 typed oracle shows that the compiler effect is not only an oracle-core artifact.

The strict action-dependent subset contains 29 goals where `hammer_empty` fails but some non-empty or non-default action succeeds. Aesop solves 20/29 strict goals and is the only successful family for 6, HammerCore solves 12/29 and is the only family for 5, and `solve_by_elim` contributes 5/29 with 2 only-family cases. This distribution rules out a Hammer-only interpretation and motivates a typed-interface portfolio.

5.3 Budgeted Allocators and Aesop Counterfactuals

Table 3 evaluates fixed and learned allocators under matched retry budgets after the shared `hammer_empty` attempt. The fixed typed portfolio is the strongest current policy: it reaches 55/230 with two retries, 57/230 with four retries out of fold, and 58/230 when the four-action

Table 3: Budgeted allocator gate after `hammer_empty`. Learned allocators match but do not beat or compress the fixed K=4 typed control.

Retry budget K	Fixed OOF	Pure logreg	Residual logreg	Residual CNB	Oracle
1	**49/230**	41/230	41/230	38/230	58/230
2	**55/230**	41/230	**55/230**	52/230	58/230
3	55/230	44/230	**56/230**	55/230	58/230
4	**57/230**	47/230	**57/230**	**57/230**	58/230

Table 4: Retrieved-only anchor after `hammer_empty`. The fixed typed portfolio reaches the retrieved-only typed oracle out of fold with three retry actions.

Retry budget K	Fixed OOF	Strict goals	Oracle
1	47/230	18/23	52/230
2	50/230	21/23	52/230
3	**52/230**	**23/23**	52/230
4	**52/230**	**23/23**	52/230

sequence is fit on all training data. Stronger text/numeric logistic and ComplementNB allocators, trained out of fold from goal text, first-failure status, and typed fact/simp pool features, do not beat or compress this control. Adaptive allocation is therefore an explicit future target, not a main claim.

Table 4 repeats the budgeted compiler test after removing traced proof-core names from the candidate source. This is the main deployability anchor for the paper: it does not reproduce a full external retriever, but it shows that once a retriever-like candidate list is fixed, typed Lean-interface allocation adds verified proofs under the same outer retry protocol.

Table 5 freezes the retrieved-only four-action subset before evaluating two fresh disjoint folds. The frozen policy is `aesop_core_plus_learned16`, `hammerCore_core_plus_learned`, `aesop_core_plus_learned_swapped`, and `aesop_core`; singleton controls are drawn from the same predeclared action subset. On the combined 432 replayable holdout goals, K=3 and K=4 solve 67 goals, compared with 54 for the best singleton and 30 for empty Hammer. The tested-action oracle is 68, so the frozen portfolio recovers nearly all available headroom without retuning on the holdout folds.

Table 5: Frozen fresh-holdout validation with retrieved-only candidates. The action subset and portfolio order are fixed before holdout evaluation; the oracle is over the predeclared tested actions, not over the full action grid.

Fold	Replayable	Empty Hammer	Best single	K=1	K=2	K=3	K=4	Oracle	Strict K=4
Fold 0	208	16	23	24	24	**28**	**28**	28	12/12
Fold 1	224	14	31	32	34	39	39	**40**	25/26
Combined	432	30	54	56	58	**67**	**67**	68	**37/38**

Table 6 checks whether the K=4 result is simply repeated use of one family. It is not: four Aesop retries reach 49/230, HammerCore-only and simplification-only reach 41/230, and Hammer-only reaches 32/230, while the typed grid reaches 57/230 out of fold. Random K=4 portfolios over the full grid average 41.9/230 over 100 seeds. The fixed typed sequence is therefore a strong control because it combines interfaces, not because it spends more calls inside one backend.

Table 7 reports empirical runner wallclock rather than treating all calls as equal compute. These numbers are not heartbeat-normalized costs, so we use them as a secondary frontier. They support the conservative wording that the main comparisons are matched by Lean-call budget, with wallclock reported for transparency.

Table 8 isolates the strongest interface under the exact P0 controls. Joint exposure remains the best Aesop configuration, but the exact controls prevent an over-strong causal interpretation: identity exposure, simps-only exposure, and single-source joint exposure account for most of the lift over

Table 6: Homogeneous K=4 controls under the same outer Lean-call budget after `hammer_empty`. Typed diversity is stronger than spending all retries inside one family.

Portfolio family	OOF K=4	Strict goals	Avg calls
Full typed grid	**57/230**	**28/29**	4.18
Aesop-only	49/230	20/29	4.23
HammerCore-only	41/230	12/29	4.34
Simplification-only	41/230	12/29	4.34
`solve_by_elim`-only	34/230	5/29	4.43
Hammer-only	32/230	3/29	4.47
Raw arithmetic/normalization	29/230	0/29	4.50

Table 7: First-success runner-wallclock frontier. The primary protocol is matched Lean calls; wallclock is reported as an empirical secondary cost.

Policy	Success	Avg calls	Avg time (s)
`hammer_empty`	29/230	1.00	8.31
`aesop_core_plus_learned` after empty	49/230	1.87	16.36
Full typed train-fitted K=2	55/230	2.66	26.37
Full typed train-fitted K=4	**58/230**	4.18	40.07

empty Aesop. The result is still useful because it shows that the evidence compiler must expose the source/channel choice explicitly, but it is no longer evidence for a large pure facts-versus-simps complementarity.

5.4 Supporting Trace-Core Discovery Evidence

The trace-core and imported-core experiments are discovery evidence rather than the verified main claim. They explain why the typed-action protocol is shaped around bounded retries and hard controls: local trace-core recovery showed that failure-conditioned feature use can improve real replay bridges, timeout stress showed that larger premise sets can be worse than one-shot retrieval, and imported-core experiments showed that local heuristics fail when proof-core premises come from global library context. The strongest bridge-facing discovery numbers are 72.0% local bridge verified success for the failure-aware controller versus 43.0% for blind expansion, and 53.5% imported bridge verified success for final-base8 versus 44.5% for learned+base fallback. We keep these results as provenance for the current design, while the main claim remains the kernel-verified typed evidence compiler in the 230-goal replay harness and frozen retrieved-only holdout folds. Appendix 10 summarizes the representative discovery numbers.

5.5 Ablations and Failure Diagnosis

The ablations in Table 9 identify why the final method is shaped as typed evidence allocation rather than a larger untyped premise list or a learned router claim. The exact Aesop controls show a useful but modest source/exposure effect, while ruling out the earlier stronger interpretation that facts and simps form a large isolated complementarity. Strict interface filtering, learned allocation gates, and the frozen holdout check jointly define the boundary: the effect survives outside the discovery matrix, but the learned-router and full-retriever claims are not yet supported.

6 Theory and Analysis

Proposition 1 (Typed interface interventions can change verified outcomes)**.** *Let $Y_g(z; B) \in \{0, 1\}$ be the kernel-verified outcome for goal g under typed evidence assignment z and search budget B. If there exist a name p and two interfaces τ_1, τ_2 such that*

$$Y_g(z + e_{p,\tau_1}; B) \neq Y_g(z + e_{p,\tau_2}; B),$$

then no interface-agnostic premise score for p can represent the full intervention effect of exposing p to Lean.

Table 8: Exact Aesop source/channel controls. Joint oracle-core plus retrieved exposure is best, but matched controls show that the mechanism is source/exposure sensitivity rather than a large pure channel-complementarity effect.

Aesop control	Verified goals	Interpretation
`empty` Aesop	29/230	default baseline
`retrieved-only`, joint	34/230	deployable-source anchor is nonzero
`oracle-core-only`, joint	35/230	oracle source alone is similar
`oracle-core+retrieved`, joint	**38/230**	best exact Aesop row
`oracle-core+retrieved`, identity	36/230	identity explains much of the lift
`oracle-core+retrieved`, simps-only	35/230	simp exposure is strong
`oracle-core+retrieved`, facts-only	32/230	fact exposure is weaker but nonzero
`oracle-core+retrieved`, random split	32/230	channel assignment still matters modestly

Table 9: Focused diagnostic results. The main verified insight is action-conditional evidence allocation; trace-core rows provide supporting discovery evidence for non-monotonic premise budgets and failure feedback.

Question	Evidence	Interpretation
Does Aesop need both exposure channels?	Exact joint: 38/230; identity: 36/230; simps-only: 35/230; facts-only: 32/230	Strong complementarity is not supported; source/exposure sensitivity remains.
Does adding more Aesop names help?	Oracle-core+retrieved8 facts+simps: 38/230; oracle-core+retrieved32 facts+simps: 3/230	Broad insertion can hurt.
Is strict syntactic filtering enough?	Filtered-only oracle: 4/230; combined oracle remains 58/230; new oracle goals: 0	Cleanup alone is not the mechanism.
Does the fixed portfolio transfer?	Fresh holdout combined: empty 30/432; best singleton 54/432; frozen K=4 67/432; oracle 68/432	The compiler effect survives a frozen retrieved-only check.
Does learned allocation beat fixed K=4?	Fixed K=4 OOF: 57/230; best learned gate: 57/230 at K=4	Adaptive routing is not a main claim.
Are visible features enough statically?	Static all-feature rerank: 86.5%; failure-conditioned: 95.7%	Failure event changes feature utility.
Which visible features matter most?	Name-only FAR-HAMMER: 95.0%; full visible FAR-HAMMER: 95.7%	Namespace/name morphology is strongest.
Is failure type alone enough?	Failure-type-only: 91.5%, equal to top-k expansion	Candidate-level conditioning is required.
Does broad base guardrail help?	Hard guardrails drop bridge trace from 72.0% to 63.0/60.0%	Broad rescue damages ranking.
Does expert mixing help?	Multi-expert bridge trace drops to 62.0%; expert-16 drops final-base8	Hand-written expert mixing is unstable.
Are misses candidate-pool failures?	200-bridge inspection: 0/17 candidate-pool misses	Remaining headroom is ranking/calibration.

Sketch. The intervention $e_{p,\tau}$ changes both the syntax sent to Lean and the backend search process that consumes the name. If two interface assignments yield different verified outcomes under the same goal and budget, a scalar premise ranking over p alone has lost information needed to choose the successful action. This is the formal reason for compiling selected evidence into typed proof actions rather than treating retrieval as the final decision.

Proposition 2 (Premise coverage is not monotonic in premise budget)**.** *There can exist premise sets $P_1 \subset P_2$ such that P_1 covers enough useful premises for the backend to succeed under budget B, while P_2 contains the same useful premises but causes timeout or reconstruction failure under the same budget.*

Sketch. Adding premises preserves proof-core coverage but can increase branching, generated clauses, simplification search, or reconstruction ambiguity. Under fixed time/search budget, the prover may fail on the larger set even though the smaller set contains the same proof-relevant lemmas. Tables 9 and 8 are empirical versions of this proposition: top-k expansion can hurt under timeout pressure, and top-32 Aesop exposure is worse than smaller typed exposure.

Proposition 3 (Complementary channels can dominate either channel alone)**.** *Suppose an interface has two evidence channels τ_f and τ_s that enable different proof-search transitions, and a goal requires transitions from both channels within budget B. Then exposing P_f only through τ_f or P_s only through τ_s can fail, while the joint typed action $(\tau_f, P_f) \cup (\tau_s, P_s)$ succeeds under the same outer retry budget.*

Sketch. The claim is existential. A fact channel can introduce the relational or propositional facts needed to close branches, while a simp channel can normalize terms or rewrite hypotheses so that those facts become applicable. If either channel is absent, search may fail to reach the closing state within budget; if both are present, the interface can interleave normalization and rule application. Table 8 is also a guardrail against overclaiming: in our exact controls, joint exposure is best at 38/230, but identity exposure reaches 36/230 and simps-only reaches 35/230, so the empirical mechanism is source/exposure sensitivity rather than a large isolated complementarity effect.

Proposition 4 (A guardrail should be narrow when the learned controller is already strong)**.** *If a learned failure-conditioned scorer improves most retry decisions but occasionally suppresses high-base-score rescue premises, then a small late-stage base guardrail can improve bridge recovery while broad base mixing can reduce success.*

Sketch. The learned controller and the base retriever encode different biases. A broad mixture changes many rankings, including rankings already corrected by failure feedback. A narrow final guardrail only affects the tail cases where high-base premises are about to be excluded. This matches the trace-core discovery pattern: final-base8 improves bridge verified success, while hard guardrails and broad expert/base mixing are harmful. It also motivates the verified typed-action protocol, where broad insertion is treated as a hypothesis to test rather than as a default improvement.

7 LIMITATIONS

The main limitation is evaluation scope. The verified typed-action results are kernel-checked, but the largest full action matrix is measured on the 230-goal replayable subset of a 490-goal cleaned trace pool, not on every Mathlib theorem and not as an open-ended theorem-proving benchmark. The fresh-holdout folds reduce the risk of post-hoc overfitting, but they are still retrieved-only compiler checks over replayable pre-theorem contexts, not a benchmark-scale population estimate. The harness patches original files in a pre-theorem context, which is the right setting for avoiding target-theorem leakage, but it also means non-replayable migrated traces are outside the current main evaluation.

A second limitation is that the current strongest policy is a fixed typed portfolio, not a learned adaptive router. The budgeted policy experiments are useful because they prevent overclaiming: NB/kNN, logistic, and ComplementNB allocators do not beat or compress the fixed typed portfolio under matched Lean-call budget. The trace-core experiments use supervised proof-core traces and should be read as discovery evidence for premise budgets and failure feedback, not as the main verified claim. Future work should either train a stronger adaptive allocator that beats the fixed typed control or expand the replayable verified corpus.

A third limitation is external comparison. The retrieved-only anchor removes traced proof-core names and supports the downstream compiler claim, but it is still not an official reproduction of LeanSearch v2 or another complete premise-selection system. The present verified result is therefore best read as orthogonal to global retrieval: once a retriever proposes names, Lean still needs a typed interface decision for how those names enter proof search.

8 CONCLUSION

This paper argues that Lean premise selection is incomplete without action-conditional evidence allocation. ACE implements this view as a typed evidence compiler over Hammer, HammerCore, simplification, `solve_by_elim`, and Aesop interfaces. In the mixed-source 230-goal matrix, the typed action grid reaches 58/230 verified proofs versus 38/230 for the best single action, and a four-retry fixed typed control recovers 57/230 out of fold. In the retrieved-only anchor, the same compiler view reaches 52/230 with a fixed portfolio, compared with 29/230 for empty Hammer and 35/230 for the best standalone action. In two frozen fresh-holdout folds, the predeclared K=4 portfolio

solves 67/432 replayable goals versus 54/432 for the best singleton and 68/432 for the tested-action oracle. The exact Aesop controls make the mechanism more precise: source and interface assignment matter, but a large pure facts-versus-simps complementarity is not supported. The result is a narrow but concrete lesson for Lean provers: retrieval should hand off not just a list of names, but a typed evidence program that tells Lean how those names should enter proof search.

REFERENCES

Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. Leansearch v2: Global premise retrieval for lean 4 theorem proving. *arXiv preprint arXiv:2605.13137*, 2026. URL https://arxiv.org/abs/2605.13137.

Thibault Gauthier, Cezary Kaliszyk, Josef Urban, Ramana Kumar, and Michael Norrish. Tactictoe: Learning to prove with tactics. *Journal of Automated Reasoning*, 65:257–286, 2021. URL https://arxiv.org/abs/1804.00596.

Edward J. Hu et al. minictx: Neural theorem proving with long contexts. *arXiv preprint arXiv:2408.03350*, 2024. URL https://arxiv.org/abs/2408.03350.

Suozhi Huang, Peiyang Song, Robert Joseph George, and Anima Anandkumar. Leanprogress: Guiding search for neural theorem proving via proof progress prediction. *arXiv preprint arXiv:2502.17925*, 2025. URL https://arxiv.org/abs/2502.17925.

Minsu Kim and Se-Young Yun. Process-verified reinforcement learning for theorem proving via lean. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=P00k4DFaXF.

David Ma, Kaijing Ma, Shawn Guo, Yunfeng Shi, Enduo Zhao, Jiajun Shi, Zhaoxiang Zhang, Gavin Cheung, Jiaheng Liu, and Zili Wang. Oprover: A unified framework for agentic formal theorem proving. *arXiv preprint arXiv:2605.17283*, 2026. URL https://arxiv.org/abs/2605.17283.

Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. Apollo: Automated llm and lean collaboration for advanced formal reasoning. In *Advances in Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=fxDCgOruk0.

Evan Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, and Vasily Ilin. Learning to repair lean proofs from compiler feedback. *arXiv preprint arXiv:2602.02990*, 2026. URL https://arxiv.org/abs/2602.02990.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2306.15626.

Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a lean hammer. *arXiv preprint arXiv:2506.07477*, 2025. URL https://arxiv.org/abs/2506.07477.

A ADDITIONAL EXPERIMENTAL DETAILS

A.1 CANONICAL OUTPUT SOURCES

All numbers in the main tables are copied from canonical JSON or Markdown outputs in the project directory:

- Verified typed action grid: outputs/mathlib430_pretheorem_action_matrix_scaled230_aesop_ablation_merged.md.
- Budgeted typed portfolio: outputs/mathlib430_budgeted_action_policy_scaled230_aesop_ablation.md.

- Typed allocator gate: outputs/mathlib430_typed_allocator_gate_scaled230_aesop_ablation.md.
- Retrieved-only anchor summary: analysis/mathlib430_retrieved_only_anchor_summary.md.
- Retrieved-only anchor matrix: outputs/mathlib430_pretheorem_action_matrix_scaled230_retrieved_only_anchor_merged.md.
- Retrieved-only fixed stability: outputs/mathlib430_fixed_portfolio_stability_scaled230_retrieved_only_anchor.md.
- Frozen fresh-holdout summary: analysis/mathlib430_fresh_holdout_summary.md.
- Fresh-holdout fold summaries: analysis/mathlib430_fresh_holdout_fold0_summary.md and analysis/mathlib430_fresh_holdout_fold1_summary.md.
- Fresh-holdout merged action outputs: outputs/mathlib430_fresh_holdout_fold0_frozen_actions_merged.md and outputs/mathlib430_fresh_holdout_fold1_frozen_actions_merged.md.
- Exact Aesop source/channel controls: analysis/mathlib430_aesop_exact_controls_summary.md.
- Initial broad Aesop fact/simp ablation: analysis/mathlib430_aesop_ablation_scaled230.md.
- Initial Aesop counterfactual channel control: analysis/mathlib430_aesop_counterfactual_controls.md.
- Evidence-contract audit and homogeneous controls: analysis/mathlib430_evidence_contract_audit.md.
- Strict interface filtering boundary: analysis/mathlib430_e1_strict_interface_filtering.md.
- Strict interface taxonomy: analysis/mathlib430_strict_action_taxonomy_scaled230_aesop_ablation.md.
- Local trace-core: outputs/phase1_mathlib_trace_core_2000_feature_ablation_a40.json.
- Phase 1 bridge: outputs/phase1_reconstruction_bridge_report_100.md.
- Timeout stress: outputs/phase2_timeout_stress_2000_a40.json.
- Imported-core heldout: outputs/phase3_second_stage_controller_eval_500_a40.json plus final guardrail deltas.
- Split stability: outputs/phase3_split_stability_summary_4x500.md.
- Imported bridge report: outputs/phase3_bridge_report_200.md.
- Imported bridge taxonomy: outputs/phase3_bridge_failure_taxonomy_200.md.
- Imported bridge inspection: outputs/phase3_bridge_negative_case_inspection_200.md.

A.2 TRACE-CORE DISCOVERY SUMMARY

Table 10: Representative trace-core discovery results that motivated the verified typed-action protocol. These rows are not the main proving metric; they explain the design pressure for bounded retries, non-monotonic premise budgets, and bridge checks.

Discovery question	Strongest useful result	Readout for current paper
Can failure feedback improve local premise recovery?	Local trace-core FAR-HAMMER reaches 95.7% trace success and 72.0% bridge verified success; blind top-k expansion reaches 91.5% trace and 43.0% bridge.	Failure events can change which evidence is useful, but bridge verification is stricter than trace-core recovery.
Can adding more premises hurt?	Timeout-conditioned shrink reaches 69.3% success with 23.5% timeout rate, while equal-budget top-k falls to 43.8% with 56.2% timeouts.	Premise coverage is not monotonic under bounded search, motivating typed budgets instead of broad insertion.
Do local heuristics solve global imported premises?	Imported-core lexical/BM25 stress stays near 12% success before learned retrieval.	Local feature control is insufficient for global retrieval; the verified paper should not claim full retriever replacement.
Does learned imported retrieval survive bridge checks?	Final-base8 averages 92.9% across four splits and reaches 53.5% imported bridge verified success versus 44.5% for learned+base fallback.	Bridge evidence supports the premise-recovery story but remains separate from open-ended proof generation.

A.3 FEATURE-GROUP ABLATION

Table 11: Feature-group ablation on 2k local trace-core goals. Static visible-feature reranking is weaker than blind expansion; failure-conditioned feature use is the important step.

Method	Success	FFR	Avg prem.
`topk_expansion`	91.5%	79.9%	46.9
`visible_feature_name_rerank`	85.3%	0.0%	42.9
`visible_feature_statement_rerank`	77.6%	0.0%	42.9
`visible_feature_decl_rerank`	78.3%	0.0%	42.9
`visible_feature_rerank`	86.5%	0.0%	42.9
`rule_far_no_core_decl_features`	92.6%	82.4%	46.2
`rule_far_no_core_statement_features`	92.8%	82.9%	46.4
`rule_far_no_core_name_features`	95.0%	88.1%	45.4
`rule_far_no_core_tags`	**95.7%**	**89.7%**	45.1

A.4 IMPORTED-CORE STRESS BEFORE LEARNED RETRIEVAL

Table 12: Initial imported-core stress test on 2k goals. This table is a diagnostic negative result: local and lexical signals are insufficient for global imported-premise recovery.

Method	Success	FFR	Avg prem.
`one_shot`	1.1%	0.0%	32.0
`same_file_prior_rerank`	2.1%	0.0%	56.0
`visible_feature_rerank`	2.4%	0.0%	56.0
`bm25_rerank`	7.0%	0.0%	56.0
`leansearch_iterative`	11.9%	8.3%	91.9
`bm25_expansion`	11.9%	8.3%	92.2
`rule_far_bm25`	**12.2%**	**8.7%**	92.2
`rule_far_full` (oracle ref.)	55.7%	55.2%	86.6

A.5 Bridge Failure Taxonomy

The 200-goal imported bridge verifies 124/200 original traced tactic scripts. The primary replay failures are 30 unknown-identifier errors, 13 other Lean errors, 8 typeclass failures, 7 unsolved-goal failures, 6 `simp` no-progress failures, 5 tactic failures, 5 type mismatches, and 2 sorry-context failures. At the policy level, final-base8 has 17 replay-verified trace misses, compared with 19 for the original second-stage controller, 35 for learned+base fallback, and 47 for learned expansion. Negative-case inspection finds 0/17 candidate-pool misses, 2 fallback-selected core drops, 3 expansion-selected core drops, and 15 failure-conditioned rank misses. This supports the limitation that remaining headroom is ranking and calibration, not candidate generation.

A.6 Guardrail and Expert-Mixing Ablations

The final base8 guardrail was selected because broader heuristics were negative. Hard base guardrails reduce bridge trace-core success from 72.0% to 63.0% or 60.0%. Soft base-score mixing is inert at low mixing weights and harmful at high weights. Imported-failure multi-expert routing drops bridge trace-core success to 62.0%. A larger final expert guardrail also underperforms final-base8 on the original split, fold0, and bridge subset. These results justify stopping hand-written expert-mix tuning and treating calibrated expert gating as future work.

B Claim-Evidence Self-Review

Table 13: End-of-draft self-review. Claims in the abstract and introduction are aligned to reported evidence; unsupported stronger claims are explicitly avoided.

Dimension	Current answer	Residual risk
Contribution	Action-conditional evidence allocation is formulated as distinct from premise ranking, with frozen holdout and Aesop controls separating the core claim from mechanism diagnostics.	A fixed portfolio can look simple unless the typed-interface intervention remains central.
Writing clarity	Main text now leads with three verified layers: mixed-source matrix, retrieved-only anchor, and frozen fresh holdout; trace-core is kept as discovery evidence.	Method names are long; final draft may need shorter aliases.
Experiments	Results cover verified replayable Mathlib actions, budgeted allocators, retrieved-only compilation, frozen fresh-holdout folds, Aesop counterfactuals, trace-core discovery, and replay bridges.	Full action grid is still 230/490; holdout is retrieved-only and not a population-scale benchmark.
Evaluation	Kernel-verified action success, oracle headroom, fixed-budget policy, holdout transfer, and interface ablations are reported.	Full open-ended theorem proving and official retriever comparison are out of scope.
Soundness	Negative gates show broad insertion, strict syntactic filtering, and learned adaptive allocation are not enough.	Learned allocation does not yet beat the fixed typed control.